База данных D

Задание 1

(a) Граф Гайфмана

Вершины — элементы домена, рёбра — между всеми парами элементов одного кортежа.

Из R_1:

- (a,b,c) → a–b, a–c, b–c
- (d,b,c) → d–b, d–c, b–c

Из R_2:

- (a,c) и (d,c) — рёбра уже есть

Из R_3:

- (f,g) → f–g
- (e,g) → e–g
- (f,h) → f–h
- (g,c) → g–c

Рёбра: {a–b, a–c, b–c, b–d, c–d, f–g, e–g, f–h, g–c}

```
      h
      |
e–g–f
   |
   c–a
   |\
   d–b
```

Граф связный: компонента {a,b,c,d} соединена с {e,f,g,h} через ребро g–c.

(b) Сферы

S_1^D(g) — соседи g: f (f–g), e (e–g), c (g–c), плюс сам g:

```
S₁^D(g) = {g, f, e, c}
```

S_2^D(e) — от e за 2 шага: на расстоянии 1 только g, на расстоянии 2 — соседи g, которых ещё нет: f, c:

```
S₂^D(e) = {e, g, f, c}
```

S_2^D((a,d)) = S_2^D(a) ∪ S_2^D(d):

- от a: dist 1 → {b,c}; dist 2 → через b: d; через c: d,g
- от d: аналогично, получается то же самое

```
S₂^D((a,d)) = {a, b, c, d, g}
```

(c) Окрестности

N_r^D(t) — подструктура D, индуцированная на S_r^D(t): берём все кортежи, все элементы которых лежат в сфере.

N_1^D(g), сфера {g, f, e, c}:

- R_1: в (a,b,c) есть a,b ∉ сфере; в (d,b,c) — d,b ∉ сфере — оба отсеиваются
- R_2: (a,c) и (d,c) — a,d ∉ сфере — отсеиваются
- R_3: (f,g) — оба в сфере; (e,g) — оба в сфере; (f,h) — h ∉ сфере; (g,c) — оба в сфере

```
N₁^D(g) = { domain: {g,f,e,c},  R₃: {(f,g),(e,g),(g,c)} }
```

N_2^D(e), сфера {e, g, f, c}:

- R_1 и R_2 содержат a,b,d — ни одного подходящего кортежа
- R_3: (f,g), (e,g), (g,c) — все в сфере; (f,h) — h ∉ сфере

```
N₂^D(e) = { domain: {e,g,f,c},  R₃: {(f,g),(e,g),(g,c)} }
```

Видно, что S_1^D(g) = S_2^D(e) = {e,f,g,c} и индуцированные подструктуры совпадают.

N_2^D((a,d)), сфера {a, b, c, d, g}:

- R_1: (a,b,c) и (d,b,c) — все в сфере
- R_2: (a,c) и (d,c) — все в сфере
- R_3: (f,g) — f ∉ сфере; (e,g) — e ∉ сфере; (f,h) — нет; (g,c) — в сфере

```
N₂^D((a,d)) = { domain: {a,b,c,d,g},
                R₁: {(a,b,c),(d,b,c)},
                R₂: {(a,c),(d,c)},
                R₃: {(g,c)} }
```

N_2^D((d,a)): сфера та же — {a,b,c,d,g}, подструктура та же. Отличается только выделенный кортеж (d,a) вместо (a,d).

(d) Эквивалентности

(e) $\approx_1$^D (g)?

- N_1^D(e): сфера {e,g}, выделен e — 2 вершины, 1 ребро
- N_1^D(g): сфера {g,f,e,c}, выделен g — 4 вершины, 3 ребра

Размеры разные, изоморфизма нет. **(e) $\not\approx_1$^D (g) — ЛОЖЬ.**

(e) $\approx_2$^D (g)?

- S_2^D(e) = {e,g,f,c} — 4 элемента
- S_2^D(g): dist 1 = {f,e,c}; dist 2 = через f: {h}; через c: {a,b,d} → итого 8 элементов

Размеры не совпадают. **(e) $\not\approx_2$^D (g) — ЛОЖЬ.**

(a,d) $\approx_2$^D (d,a)?

Обе сферы одинаковы: {a,b,c,d,g}, подструктуры тоже.

Проверим автоморфизм σ: a↔d, остальные фиксированы:

- R_1: (a,b,c) ↦ (d,b,c) ✓ и обратно ✓
- R_2: (a,c) ↦ (d,c) ✓ и обратно ✓

- R_3: (g,c) ↦ (g,c) ✓

(a,d) $\approx_2$^D (d,a) — ИСТИНА.

(e) * Верно ли, что любой SQL-запрос, выбирающий (a), выбирает и (d)?

ИСТИНА.

σ: a↔d, всё остальное фиксировано — это автоморфизм всей базы D:

- R_1: {(a,b,c),(d,b,c)} → {(d,b,c),(a,b,c)} — то же множество
- R_2: {(a,c),(d,c)} → {(d,c),(a,c)} — то же
- R_3: не затрагивается

SQL-запросы инвариантны относительно автоморфизмов: если D ⊨ ϕ(a), то D ⊨ ϕ(d). Никакой SQL-запрос не может различить a и d в этой базе.

Задание 2: Рекурсивный SQL для Middle Element и Cycle

Middle Element

Ищем тройки (a_i, a_j, a_k), где a_j строго посередине: dist(a_i,a_j) = dist(a_j,a_k).

```
WITH RECURSIVE
  reach(x, y, dist) AS (
    SELECT a, a, 0 FROM (SELECT a FROM E UNION SELECT b FROM
E) AS N(a)
    UNION ALL
    SELECT r.x, e.b, r.dist + 1
    FROM reach r
    JOIN E e ON r.y = e.a
  )
SELECT r1.x AS a_i, r1.y AS a_mid, r2.y AS a_k
FROM reach r1
JOIN reach r2
  ON r1.y = r2.x
 AND r1.dist = r2.dist;
```

Cycle

Узлы, лежащие на цикле — те, из которых можно вернуться в себя.

```sql
WITH RECURSIVE
  reach(start, current) AS (
    SELECT a, b FROM E
    UNION
    SELECT r.start, e.b
    FROM reach r
    JOIN E e ON r.current = e.a
  )
SELECT DISTINCT start AS cycle_node
FROM reach
WHERE start = current;
```

Задание 3: Нет RA-запроса для Middle Element

Предположим, такой Q существует. По теореме Гайфмана (Hanf) любой RA-запрос r-локален для некоторого r. Возьмём цепочку R_{2l} при l > 2r+1.

Рассмотрим две тройки:

- **t_1 = (a_r, a_l, a_{2l−r})** — входит в Q(R_{2l}), так как l−r = (2l−r)−l
- **t_2 = (a_r, a_{l+1}, a_{2l−r})** — не входит, так как (l+1)−r ≠ (2l−r)−(l+1)

Их r-окрестности совпадают: крайние элементы a_r и a_{2l−r} видят по r шагов в каждую сторону, а средние a_l и a_{l+1} при l > 2r+1 тоже видят по r шагов, не достигая ни концов, ни друг друга. Все три «куска» изоморфны в обоих случаях.

По r-локальности Q должен давать одинаковый ответ на t_1 и t_2 — противоречие. Значит, Q не существует. □

Задание 4: Ранг локальности запроса Q(R)

$$Q(R) := R \cup (\pi_{$1,$4}, \sigma_{$2=$3}(R \times R))$$

Если R — бинарное отношение (A, B), то R × R имеет атрибуты ($1,$2,$3,$4), условие $2=$3 отбирает строки где B=A′, проекция на ($1,$4) даёт пары (A,B′) — это R^2 (двухшаговая достижимость). Итого **Q(R) = R ∪ R^2**.

(a,b) ∈ Q(D) когда либо (a,b) ∈ R напрямую, либо существует c с (a,c) ∈ R и (c,b) ∈ R.

Во втором случае c — сосед a в графе Гайфмана, т.е. c ∈ S_1^D({a,b}), и оба кортежа (a,c),(c,b) лежат в N_1^D({a,b}).

- r=0 не работает: при r=0 видим только {a,b}, промежуточный c недоступен
- r=1 достаточно: все такие c входят в S_1^D({a,b})

Ранг локальности Q равен 1.